Module 6 -- Robotics Projects

Lesson 45: Line-Following Robot

★ Star-5 Expert | ⏰ 50--60 minutes | 👤 Age 6--12 years | ⚡ 350 XP

Your Mission Today

Robot Engineer, today your robot gets its very first sense — SIGHT! Well, sort of. You are going to add **IR line sensors** to the bottom of your robot so it can see a black line on a white surface. Then you will program it to FOLLOW that line, all by itself, with NO remote control. This is called **autonomous navigation** — the robot makes its own decisions. Welcome to real robotics!

Learning Objectives

By the end of this lesson, you will be able to:

- Explain how IR line sensors detect dark and light surfaces
- Mount and wire IR sensors to your robot
- Write a line-following algorithm using if-else logic
- Build a track and tune the robot to follow it smoothly
- Use your Magic Measurement Wand to measure IR sensor output voltage on black vs white surfaces

What You Need

Item	Qty
Your assembled robot from Lessons 43-44	1
IR line sensor modules (TCRT5000 or similar)	2
Jumper wires (female-to-male)	6
White poster board or large white paper	1
Black electrical tape (2 cm wide)	1 roll
Multimeter (your Magic Measurement Wand!)	1
Small screws or hot glue (for mounting sensors)	-
USB cable (for programming)	1

How to Teach This Lesson

Step 1: Hook — How Does a Robot See a Line? (5 min)

Put a strip of black tape on white paper in front of the kid.

💬

> "Look at this line. Your eyes can see it easily — black on white. But how would a ROBOT see it? A robot does not have eyes like you. Instead, it uses a special sensor that shines invisible light at the ground and watches what bounces back."

Hold up an IR sensor module:

> 💬 "This tiny sensor is like a flashlight and a camera in one. It shines INFRARED light (which you cannot see) at the ground. White surfaces REFLECT the light back, and the sensor says HIGH. Black surfaces ABSORB the light, and the sensor says LOW. It is like shining a flashlight at a mirror versus a black hole!"

```
How IR Line Sensors Work:

White Surface (reflects):       Black Surface (absorbs):

SENSOR                          SENSOR
  |  \    /  |                    |  \       |
  |   \  /   |                    |   \      |
  |    \/    |                    |    \     |
============== (white)            |||||||||| (black)
  IR bounces back!                IR gets absorbed!
  Output: HIGH (1)                Output: LOW (0)
```

Award: **+10 XP** **for understanding how IR sensors work!**

Step 2: Sensor Wiring (8 min)

Each IR sensor module has 3 pins: VCC, GND, and OUT (signal).

Wiring:

```
Left IR Sensor:                 Right IR Sensor:
VCC ---> Arduino 5V             VCC ---> Arduino 5V
GND ---> Arduino GND            GND ---> Arduino GND
OUT ---> Arduino Pin 7           OUT ---> Arduino Pin 8
```

Mounting Position:

Mount both sensors on the bottom of the robot, at the FRONT, about 1.5—2 cm above the ground, spaced apart just wider than the black tape:

```
Front of Robot (bottom view):

+----------------------------+
|                            |
|    [Left      [Right       |
|     Sensor]    Sensor]     |
|       ^           ^        |
|       |           |        |
|       +--- gap ---+        |
|      (tape width)          |
|                            |
|   Direction of travel -->  |
+----------------------------+
```

💬

"The two sensors should be positioned so that when the robot is centered on the tape, BOTH sensors see the tape (black) or BOTH sensors see white on either side. When the robot drifts, one sensor sees black and the other sees white — that is how it knows to correct!"

Award: **+20 XP** **for mounting and wiring the sensors!**

Step 3: Testing the Sensors (5 min)

Before programming line-following, let us make sure the sensors work:

```
// Lesson 45: IR Sensor Test
int leftSensor = 7;
int rightSensor = 8;

void setup() {
  pinMode(leftSensor, INPUT);
  pinMode(rightSensor, INPUT);
  Serial.begin(9600);
  Serial.println("IR Sensor Test");
}

void loop() {
  int L = digitalRead(leftSensor);
  int R = digitalRead(rightSensor);

  Serial.print("Left: ");
  Serial.print(L);
  Serial.print("  Right: ");
  Serial.print(R);

  if (L == LOW && R == LOW) Serial.println("  -> Both on BLACK");
  else if (L == HIGH && R == HIGH) Serial.println("  -> Both on WHITE");
  else if (L == LOW) Serial.println("  -> Left on BLACK");
  else Serial.println("  -> Right on BLACK");

  delay(200);
}
```

Test it:

1. Hold the robot above the white paper — both sensors should read HIGH
2. Hold it above the black tape — both sensors should read LOW
3. Position it so one sensor is on tape, one on white — they should read differently

> "If both sensors always read the same, adjust the sensor height or check the tiny potentiometer on the sensor module — it adjusts sensitivity."

Award: **+20 XP** **for verifying both sensors work correctly!**

Step 4: The Line-Following Algorithm (10 min)

Here is the brain of your line-following robot:

```
Decision Table:

Left Sensor | Right Sensor | Meaning           | Action
------------|-------------|-------------------|--------
HIGH (white)| HIGH (white)| Both ON the line  | Go FORWARD
LOW (black) | HIGH (white)| Drifting RIGHT    | Turn LEFT
HIGH (white)| LOW (black) | Drifting LEFT     | Turn RIGHT
LOW (black) | LOW (black) | LOST the line!    | STOP (search)
```

💬

"Wait — that might seem backwards! If the LEFT sensor sees black (off the line), it means the robot drifted to the RIGHT. So it needs to turn LEFT to get back on track. Think of it like this: the sensor that sees black is saying 'HELP! I fell off the road!' and the robot steers AWAY from that side."

Note on sensor polarity: Some IR sensor modules output HIGH for black and LOW for white (inverted). Check your sensors during the test in Step 3. If your sensors are inverted, swap HIGH and LOW in the code below.

The Complete Line-Following Code:

```
// Lesson 45: Line-Following Robot
// Uses 2 IR sensors + movement functions from Lesson 44

// Motor pins
int leftENA = 9;
int leftIN1 = 2;
int leftIN2 = 3;
int rightENB = 10;
int rightIN3 = 4;
int rightIN4 = 5;

// Sensor pins
int leftSensor = 7;
int rightSensor = 8;

// Speed settings
int driveSpeed = 160;   // Speed when going straight
int turnSpeed = 140;    // Speed when correcting

void setup() {
  // Motor pins
  pinMode(leftENA, OUTPUT);
  pinMode(leftIN1, OUTPUT);
  pinMode(leftIN2, OUTPUT);
  pinMode(rightENB, OUTPUT);
  pinMode(rightIN3, OUTPUT);
  pinMode(rightIN4, OUTPUT);

  // Sensor pins
  pinMode(leftSensor, INPUT);
  pinMode(rightSensor, INPUT);

  Serial.begin(9600);
  Serial.println("Line Follower Ready!");
  delay(3000);  // 3 second countdown to place on track
}

void loop() {
  int L = digitalRead(leftSensor);
  int R = digitalRead(rightSensor);

  if (L == HIGH && R == HIGH) {
    // Both on line -- go straight!
    forward(driveSpeed);
  }
```

```
    // Left off line -- robot drifted right -- turn left
    turnLeft(turnSpeed);
  }
  else if (L == HIGH && R == LOW) {
    // Right off line -- robot drifted left -- turn right
    turnRight(turnSpeed);
  }
  else {
    // Both off line -- lost! Stop and search.
    stopRobot();
  }

  delay(10);  // Small delay for stability
}

// Movement functions (from Lesson 44)
void forward(int speed) {
  digitalWrite(leftIN1, HIGH);
  digitalWrite(leftIN2, LOW);
  digitalWrite(rightIN3, HIGH);
  digitalWrite(rightIN4, LOW);
  analogWrite(leftENA, speed);
  analogWrite(rightENB, speed);
}

void turnLeft(int speed) {
  digitalWrite(leftIN1, LOW);
  digitalWrite(leftIN2, LOW);     // Left wheel STOP
  digitalWrite(rightIN3, HIGH);
  digitalWrite(rightIN4, LOW);    // Right wheel FORWARD
  analogWrite(leftENA, 0);
  analogWrite(rightENB, speed);
}

void turnRight(int speed) {
  digitalWrite(leftIN1, HIGH);
  digitalWrite(leftIN2, LOW);     // Left wheel FORWARD
  digitalWrite(rightIN3, LOW);
  digitalWrite(rightIN4, LOW);    // Right wheel STOP
  analogWrite(leftENA, speed);
  analogWrite(rightENB, 0);
}

void stopRobot() {
  digitalWrite(leftIN1, LOW);
```

```
  digitalWrite(rightIN4, LOW);
}
```

💬

"Notice that the turnLeft and turnRight functions here are different from Lesson 44! Instead of spinning in place, we STOP one wheel and keep the other going. This makes GENTLE corrections perfect for line following."

Award: **+40 XP** **for uploading the line-following code!**

Step 5: Build the Track (5 min)

Create a track using black electrical tape on white poster board or a white floor:

```
Track Layout Ideas:

1. Simple Oval:         2. Figure-8:         3. With Sharp Turns:
+----------+           +---+  +---+          +--+
|          |           |   |  |   |          |  |
|          |           |   +--+   |          |  +--+
|          |           |          |          |     |
+----------+           +---+  +---+          +--+  |
                           |  |                  |  |
                           +--+                  +--+
```

Tips for building the track:

- Press the tape down firmly — no bubbles or lifted edges
- Start with gentle curves — sharp 90-degree corners are HARD
- Make the track at least 2 cm wide
- Ensure the surface under the track is white or very light colored

Award: **+20 XP** **for building a track!**

Step 6: Tune and Test (8 min)

Place the robot on the track and let it run!

Tuning Guide:

Problem	Fix
Robot overshoots curves	Reduce driveSpeed (try 140)
Robot wobbles side to side	Reduce turnSpeed or add a small delay
Robot loses the line on sharp turns	Slow down or widen the tape at corners
Robot does not respond to line	Check sensor height (1.5—2cm from ground)
Robot goes the wrong way at curves	Swap left and right sensor pin assignments

> "Tuning a robot is one of the most important skills in robotics. Professional robots go through hundreds of test runs! Be patient and keep adjusting."

Award: **+30 XP** **for a successful line-following run (at least one full lap)!**

Step 7: Wand Check — IR Sensor Output Voltage (8 min)

> "Let us use your Magic Measurement Wand to see EXACTLY what voltage your IR sensors output on different surfaces!"

Set the Wand to **DC Volts**. Measure the OUT pin of each sensor (relative to GND):

Test 1: Output voltage on different surfaces

```
| Surface         | Left Sensor Voltage | Right Sensor Voltage |
|----------------|--------------------|--------------------|
| White paper     |                    |                    |
| Black tape      |                    |                    |
| Gray surface    |                    |                    |
| Wooden table    |                    |                    |
```

Expected results:

- White surface: 4—5V (HIGH)
- Black tape: 0—0.5V (LOW)
- Gray surface: somewhere in between (this is why gray can confuse robots!)

💬

> "Your Wand shows you the EXACT voltage the Arduino sees. When the sensor says HIGH, Arduino reads about 5V. When it says LOW, Arduino reads almost 0V. The Wand just showed you the invisible conversation between the sensor and the Arduino!"

Bonus measurement: Measure the sensor VCC pin to confirm it is getting a full 5V from Arduino.

Award: **+50 XP** **for all sensor Wand measurements!**

Quick Quiz — Earn Bonus XP!

Question 1: How does an IR line sensor tell the difference between black and white?

- A) It looks at the color with a camera
- B) It shines infrared light and detects how much bounces back
- C) It measures temperature

(Correct: B — **+20 XP** **!)**

Question 2: If the LEFT sensor sees black and the RIGHT sensor sees white, which way should the robot turn?

- A) Turn LEFT to get back on track
- B) Turn RIGHT

(Correct: A — +20 XP !)

Question 3: Your Wand reads 4.8V on the sensor output over white paper and 0.2V over black tape. What digital values does Arduino see?

- A) White = LOW, Black = HIGH
- B) White = HIGH, Black = LOW
- C) Both = HIGH

(Correct: B — +20 XP !)

Question 4: The robot keeps overshooting curves. What should you try?

- A) Use bigger batteries
- B) Reduce the drive speed
- C) Remove one sensor

(Correct: B — +20 XP !)

Lesson 45 Complete!

```
==========================================

   LINE TRACKER BADGE UNLOCKED!

   Skills unlocked:
   [check] Wired and tested IR line sensors
   [check] Understood line-following logic
   [check] Built a track
   [check] Programmed autonomous line following
   [check] Tuned robot for smooth operation
   [check] Measured sensor voltages with Wand

==========================================
```

XP Breakdown:

Activity	XP
Hook understanding	10
Sensor mounting + wiring	20
Sensor testing	20
Line-following code	40
Track building	20
Successful line-following run	30
Wand Check (sensor voltages)	50
Quiz (4 questions)	80
TOTAL POSSIBLE	**270**

Coming Up Next...

In **Lesson 46**, your robot gets a whole new superpower — **obstacle avoidance**! You will mount an ultrasonic distance sensor on the front and a servo to scan left and right. Your robot will be able to navigate around obstacles like a bat using echolocation!

Troubleshooting

Problem	Fix
Both sensors always read HIGH	Sensors too far from ground — lower them to 1.5—2cm
Both sensors always read LOW	Tape too reflective or sensor sensitivity too high — adjust onboard potentiometer
Robot follows line but wobbles a lot	Reduce turn speed or use proportional control
Robot loses the line at curves	Slow down the drive speed or make curves more gradual
Sensor LED does not light up	Check VCC and GND wiring — swap if reversed
Wand reads 2.5V on both surfaces	Sensor sensitivity needs adjustment — turn the onboard potentiometer